

ЛЕКЦИЯ 7

Обучение с подкреплением в распределённых робототехнических системах

От уравнений Беллмана до многоагентных политик, которые работают на железе

две части по 45

«Обучение распределёнными робототехническими системами»

План лекции

1 Постановка задачи: агент, среда, вознаграждение

2 MDP и уравнения Беллмана

3 Q-обучение, SARSA, стратегии исследования

4 Приближение функций и глубокое Q-обучение

5 Методы градиента политики

6 Многоагентность: Dec-POMDP и две её беды

7 CTDE: главная парадигма и её границы

8 VDN, QMIX, MADDPG, IPPO, MAPPO

9 Воспроизводимость, бенчмарки, sim-to-real

10 Применения в PPTC, масштабируемость, безопасность

Синие — первая часть · янтарные — вторая часть

ЧАСТЬ 1

Часть 1. Обучение с подкреплением: один агент

MDP, уравнения Беллмана, Q-обучение и SARSA, приближение функций и градиент политики.

1

Зачем это нужно: когда правила писать нечем

- До сих пор стратегия задавалась разработчиком явно: правила, протоколы, законы управления
- Среда бывает заранее неизвестна, изменчива или слишком сложна для анализа
- Аналитический синтез управления становится неоправданно трудоёмким
- Альтернатива: агент сам формирует стратегию из опыта взаимодействия
- В прошлой лекции про рой мы подвесили обратную задачу — здесь один из ответов

1

Цикл взаимодействия агента со средой



Ключевое отличие от обучения с учителем: сигнал оценивающий, а не указывающий.

1

Компоненты задачи

- **Агент** — обучающаяся система управления робота
- **Среда** — всё вне агента: физика, объекты, другие роботы
- **S** — множество состояний, **A** — множество действий
- **R** — скалярная оценка; функция вознаграждения кодирует цель системы
- $\pi(a|s)$ — политика, в общем случае стохастическая

Ключевая идея

Вознаграждение задаёт цель, но не способ. Плохо спроектированная функция вознаграждения даёт формально успешную политику, делающую совсем не то, чего от неё ждали.

Две фундаментальные трудности

- **Отложенное вознаграждение:** последствия проявляются через десятки шагов
- Неудачный манёвр в начале маршрута приводит в тупик много тактов спустя
- **Исследование против использования:** применять знания или пробовать новое
- Слишком мало исследования — застреваем на посредственной стратегии
- Слишком много — агент вечно экспериментирует и не собирает награду

$$G_t = r_{t+1} + \gamma \cdot r_{t+2} + \gamma^2 \cdot r_{t+3} + \dots = \sum_k \gamma^k \cdot r_{t+k+1}$$

- MDP задаётся кортежем $\langle S, A, P, R, \gamma \rangle$
- $P(s' | s, a)$ — вероятности переходов, $R(s, a)$ — ожидаемое вознаграждение
- $\gamma \in [0, 1]$ — дисконт: при $\gamma < 1$ бесконечная сумма сходится
- Задача: найти π^* , максимизирующую $E[G_t]$ из любого начального состояния

Максимизируется накопленная доходность, а не награда текущего шага.

Дисконт, марковское свойство и реальность

- $\gamma \rightarrow 0$: агент близорук, максимизирует немедленную выгоду
- $\gamma \rightarrow 1$: почти равный вес отдалённым последствиям; в робототехнике 0,9–0,99
- **Марковское свойство**: будущее зависит только от текущего состояния и действия
- У робота оно выполняется приближённо: сенсоры шумят и видят не всё
- Честная модель — частично наблюдаемый MDP (POMDP)



Как думаете?

Складской робот учится доставлять грузы; эпизод длится около 200 шагов, награда за доставку приходит в самом конце. Разработчик поставил $\gamma = 0,8$. Что произойдёт?

A

Ничего страшного, 0,8 — стандартное значение

B

Финальная награда почти не влияет на ранние действия, обучение застрянет

C

Агент переобучится и будет ехать слишком быстро

D

Нарушится марковское свойство

Ответ: Награда обесценится: $0,8^{200}$ — практически ноль, ранние состояния о цели не узнают. Для длинных эпизодов нужно γ около 0,99 или промежуточные награды.

$$V\pi(s) = \sum_a \pi(a|s) \cdot \sum_{s'} P(s'|s,a) \cdot [R(s,a) + \gamma \cdot V\pi(s')]$$

- $V\pi(s) = E_{\pi}[G_t \mid s_t = s]$ — ценность состояния при политике π
- $Q\pi(s, a) = E_{\pi}[G_t \mid s_t = s, a_t = a]$ — ценность действия в состоянии
- Смысл: ценность = немедленная награда + дисконтированная ценность следующего состояния
- Усреднение идёт по действиям политики и по стохастике среды

$$Q^*(s,a) = \sum_{s'} P(s'|s,a) \cdot [R(s,a) + \gamma \cdot \max_{a'} Q^*(s',a')]$$

- В уравнении оптимальности усреднение по политике заменено максимумом
- $\pi(s) = \operatorname{argmax}_a Q(s, a)$ — оптимальная политика получается мгновенно
- Если P и R известны — динамическое программирование (итерация ценности и политики)
- В робототехнике модель среды недоступна → учимся прямо на опыте

Именно поэтому большая часть алгоритмов занята оценкой Q^* .

“

Знать оптимальную Q-функцию — значит знать оптимальное поведение: достаточно в каждом состоянии брать максимум

Почему полкурса RL — это оценивание Q

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$

- Уоткинс, 1989; $\alpha \in (0, 1]$ — скорость обучения
- Выражение в скобках — TD-ошибка: расхождение оценки и её улучшенной версии
- **Off-policy**: в цели стоит максимум, а не фактически выбранное действие
- Сходится к Q^* при бесконечном посещении всех пар (s, a) и убывании α

3

Эпизод Q-обучения с ϵ -жадным выбором

Весь табличный алгоритм — около 15 строк кода; он же ядро всех дальнейших усложнений.

3 Сеточный мир: как награда просачивается назад

Оценка	Старт	Эпизод 1	Эпизод 2	Эпизод 3	Q^*
$Q(s_1, \Pi)$	0	-0,50	-0,75	-0,20	6,2
$Q(s_2, \Pi)$	0	-0,50	1,50	4,03	8,0
$Q(s_3, \Pi)$	0	5,00	7,50	8,75	10,0

Коридор $s_1-s_2-s_3-s_4$, шаг -1, цель +10, $\alpha = 0,5$, $\gamma = 0,9$.

Q-обучение (off-policy)

- Цель: $r + \gamma \cdot \max_{a'} Q(s', a')$
- Оценивает оптимальную политику
- Игнорирует риски исследования
- Обучение в симуляции, чужой опыт

SARSA (on-policy)

- Цель: $r + \gamma \cdot Q(s', a')$ по факту
- Оценивает политику с исследованием
- Осторожнее обходит опасные зоны
- Реальный робот, цена ошибки высока

- **ε-жадная**: с вероятностью ε случайное действие, иначе жадное
- ε уменьшают по ходу обучения: сначала оценки ненадёжны
- Недостаток: при исследовании плохие и почти оптимальные действия равновероятны
- **Softmax (Больцман)**: $\pi(a|s) = \exp(Q(s,a)/\tau) / \sum_b \exp(Q(s,b)/\tau)$
- τ большое — почти равновероятно, τ малое — почти жадно

4

Почему таблица не масштабируется

- Состояние робота непрерывно: координаты, скорости, дальномеры, изображения
- Число различных состояний астрономически велико — таблицу не построить
- Выход: параметрическая модель $Q(s, a; \theta)$ и обучение параметров θ
- Линейное приближение $Q = \theta^T \cdot \phi(s, a)$: дёшево, есть гарантии, но нужны признаки вручную
- Нейросеть снимает ручное конструирование признаков — ценой устойчивости

$$L(\theta) = E[(r + \gamma \cdot \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2]$$

- Мних и соавторы, 2015: Q-функция аппроксимируется глубокой сетью
- θ^- — параметры целевой сети, замороженные и обновляемые периодически
- Обучение — минимизация квадрата той же TD-ошибки
- Развитие: Double DQN, Dueling DQN, приоритетное воспроизведение

Два механизма устойчивости DQN

- **Буфер воспроизведения:** переходы (s, a, r, s') складываются в память
- Обучение идёт на случайных мини-выборках — корреляции соседних шагов разрушаются
- Бонус: опыт используется многократно, а он в робототехнике дорог
- **Целевая сеть:** θ^- заморожены, иначе цель убегает вместе с оценкой
- Без обоих механизмов наивное соединение Q-обучения и сети расходится

Ключевая идея

Погоня за движущейся мишенью — самая частая причина расходимости. Замораживание цели на несколько тысяч шагов превращает нестабильную задачу в почти обычную регрессию.

- Политика оптимизируется напрямую: $\pi(a|s; \theta)$, $\theta \leftarrow \theta + \alpha \cdot \nabla J(\theta)$
- $\nabla J(\theta) = E[\nabla \ln \pi(a|s; \theta) \cdot Q\pi(s,a)]$ — усиливаем удачные действия
- Естественно работают с **непрерывными** действиями: усилия, скорости, углы
- **Актор — критик**: актор держит политику, критик оценивает ценность
- REINFORCE, PPO, DDPG — DDPG станет основой многоагентного MADDPG

П Е Р Е Р Ы В

Конец первой части

Разобрали MDP, Беллмана, Q-обучение и SARSA, DQN и градиент политики. После перерыва — обучение коллектива роботов и то, почему это совсем другая задача.

ЧАСТЬ 2

Часть 2. Многоагентное обучение

Дес-POMDP, нестационарность и распределение заслуг, парадигма CTDE, современные методы и честный разговор о воспроизводимости и переносе на железо.

Что меняется, когда агентов много

- Состояние и вознаграждение зависят от **совместного** действия всех агентов
- Каждый робот наблюдает лишь локальную часть обстановки
- Интересы совпадают, противоречат или смешаны — кооперация, конкуренция, смесь
- Одноагентные гарантии сходимости перестают действовать
- Стандартная модель кооперативного случая — Dec-POMDP

$$\langle N, S, \{A_i\}, P, R, \{\Omega_i\}, O, \gamma \rangle$$

- N — множество из n агентов; совместное действие $a = (a_1, \dots, a_n)$
- $R(s, a)$ — общее командное вознаграждение, одно на всех
- Ω_i, O — локальные наблюдения агента и вероятности их получения
- Точное решение NEXP-полно $\rightarrow$ только приближённые обучаемые методы

Каждый агент строит политику по истории лишь собственных наблюдений и действий.

Беда первая: нестационарность

- Для агента остальные роботы — часть среды
- Пока они учатся, их политики меняются, а с ними и «динамика среды»
- Марковское свойство нарушается: вчерашняя оценка сегодня неверна
- Причина ошибки — не шум среды, а изменившееся поведение соседа
- Гарантии сходимости одноагентных алгоритмов теряются

Беда вторая: распределение заслуг

- При общем вознаграждении все получают один и тот же скаляр
- Из него не видно, чей вклад был полезным, а чей вредным
- Робот, простоявший весь эпизод, получит столько же, сколько работавший
- Результат — «ленивые агенты» и медленное, ненадёжное обучение
- Лечится индивидуальными штрафами, декомпозицией ценности, контрфактикой

Пример

Команда из пяти роботов получила +100 за собранный заказ. Один из них весь эпизод простоял в углу — и его политика получила ровно то же положительное подкрепление, что и у работавших.



Как думаете?

Два робота учатся независимо по Q-обучению. Кривые обучения идут вверх, потом резко падают, потом снова растут — и так по кругу. В чём вероятная причина?

A

Слишком большая скорость обучения α

B

Нестационарность: агенты гоняются за меняющимися политиками друг друга

C

Недостаточный размер буфера воспроизведения

D

Неверно выбран коэффициент дисконтирования γ

Ответ: Нестационарность. Циклические колебания — характерная подпись независимого обучения: агенты подстраиваются друг под друга по кругу и не сходятся.

6 Два полюса: независимое и совместное обучение

Независимое обучение (IQL)

- У каждого свой одноагентный алгоритм
- Полная децентрализация, линейный рост
- Нестационарность, нет гарантий
- Рабочая базовая линия при слабой связности

Обучение по совместным действиям

- Оценивается $Q(s, a_1, \dots, a_n)$
- Стационарность восстановлена
- m^n комбинаций: 10 роботов $\times$ 5 действий $\approx 10^7$
- Исполнение требует связи всех со всеми

7 CTDE: централизованное обучение, децентрализованное исполнение

- Обучение и эксплуатация — разные режимы, и это можно использовать
- При обучении (симулятор, полигон) доступна глобальная информация
- При исполнении каждый робот опирается только на локальные наблюдения
- Нестационарность снимается: централизованный обучатель «видит» всех
- Децентрализованность итоговой системы при этом сохраняется

Ключевая идея

Глобальное состояние — это не то, что робот должен иметь в эксплуатации. Это ресурс, который есть у нас на этапе обучения, и грех им не воспользоваться.

- Нужен симулятор или полигон с доступом к глобальному состоянию
- Вход централизованного критика растёт с числом агентов
- Монотонная декомпозиция ценности выразительна не всегда
- Обучение под фиксированный состав команды: сменился n — политика деградирует
- Партнёры на исполнении могут отличаться от тех, с кем учились

$$Q_{\text{tot}} = \sum_i Q_i(o_i, a_i) \rightarrow \partial Q_{\text{tot}} / \partial Q_i \geq 0$$

- VDN: командная ценность — сумма индивидуальных полезностей
- QMIX (Рашид и др., 2018): вместо суммы — смешивающая сеть с неотрицательными весами
- Веса микшера генерируются по глобальному состоянию
- Монотонность даёт согласованность argmax : локально жадное = глобально лучшее

Обучается Q_{tot} по командной награде, а выполняется каждая Q_i отдельно.

Централизованный критик: MADDPG и COMA

- **MADDPG** (Лоу и др., 2017): актер $\pi_i(o_i)$ локален, критик $Q_i(x, a_1, \dots, a_n)$ — централизован
- Критик видит действия всех, поэтому его задача стационарна
- На исполнении критики отбрасываются — работают только акторы
- Годится и для соревновательных и смешанных задач: у каждого свой критик
- **COMA** (Фёрстер и др., 2018): контрфактическая база «а что, если бы я поступил иначе»

IPPO и MAPPO: простое побеждает сложное

- **IPPO** — независимый PPO: каждый агент учит свою политику, никакой централизации
- **MAPPO** — тот же PPO с централизованной функцией ценности (критик видит состояние)
- Оба на многих бенчмарках обходят QMIX, MADDPG и более сложные схемы
- Причина не в теории, а в инженерии: PPO устойчив и хорошо переносит настройку
- Практический вывод: сложный метод обязан доказать превосходство над IPPO/MAPPO

Факт

Независимое обучение, которое годами считали лишь наивной базовой линией, в версии на PPO оказалось конкурентоспособным с методами CTDE. Это один из главных сюжетов области за последние годы.

Метод	Что обучается	Информация при обучении	Слабое место
IQL	$Q_i(o_i, a_i)$ у каждого	только локальная	нестационарность
VDN	$Q_{\text{tot}} = \sum Q_i$	командная награда	аддитивность слишком груба
QMIX	монотонная смесь Q_i	глобальное состояние в микшере	ограничение монотонности
MADDPG	акторы + критики по всем действиям	наблюдения и действия всех	рост входа критика
COMA	акторы + контрфактический критик	глобальное состояние	стоимость контрфактики
IPPO / MAPPO	политики на PPO	нет / глобальное состояние	чувствительность к гиперпараметрам

Строки 2–5 — семейство CTDE; MAPPO тоже CTDE, но с минимальной централизацией.

- Результаты сильно зависят от реализации, а не только от идеи метода
- Разные статьи мерят на разных средах, с разным бюджетом шагов и числом запусков
- Малое число случайных сидов даёт красивые, но невоспроизводимые графики
- Ответ сообщества — стандартизованные наборы бенчмарков и единые протоколы
- SMAC и SMACv2, MPE, PettingZoo, Google Research Football, Melting Pot, Hanabi

Ключевая идея

Сравнение методов честно только при одинаковом бюджете взаимодействий, одинаковой процедуре настройки гиперпараметров и достаточном числе независимых запусков с доверительными интервалами.

$10^6 - 10^8$

шагов взаимодействия на одну обученную
политику

≥ 5

независимых запусков нужно для честного
сравнения

2024

первый полноценный учебник по
многоагентному RL, MIT Press

- Такой объём достижим только в симуляции с параллельными средами на GPU
- На реальном роботе миллион эпизодов физически невозможен
- Отсюда единственный рабочий сценарий: учим в симуляторе, переносим на железо

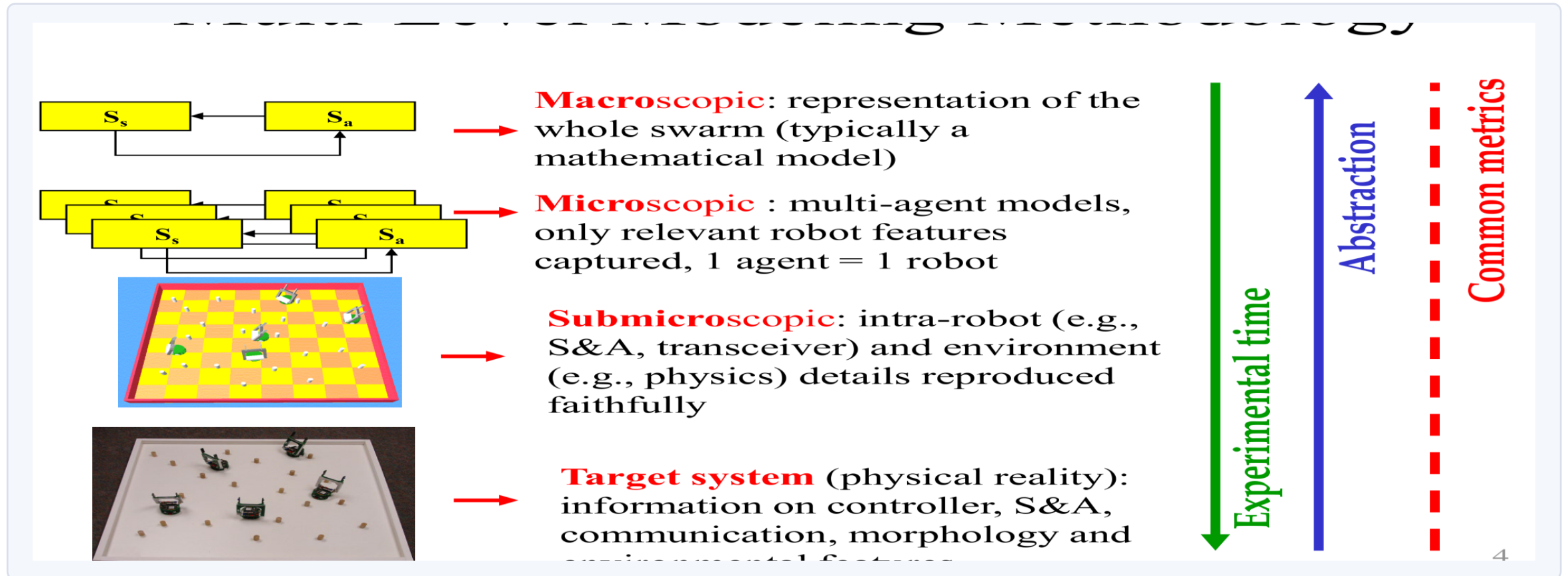
Разрыв «симуляция — реальность»

- Политика, отличная в симуляторе, деградирует на реальном роботе
- Причины: неточная физика, шум и задержки сенсоров, износ, реальная связь
- В многоагентном случае разрыв больше: ошибки партнёров накапливаются
- **Рандомизация** параметров симулятора — учим на распределении миров, а не на одном
- Плюс обучение на разнообразных средах и дообучение на объекте

Кейс

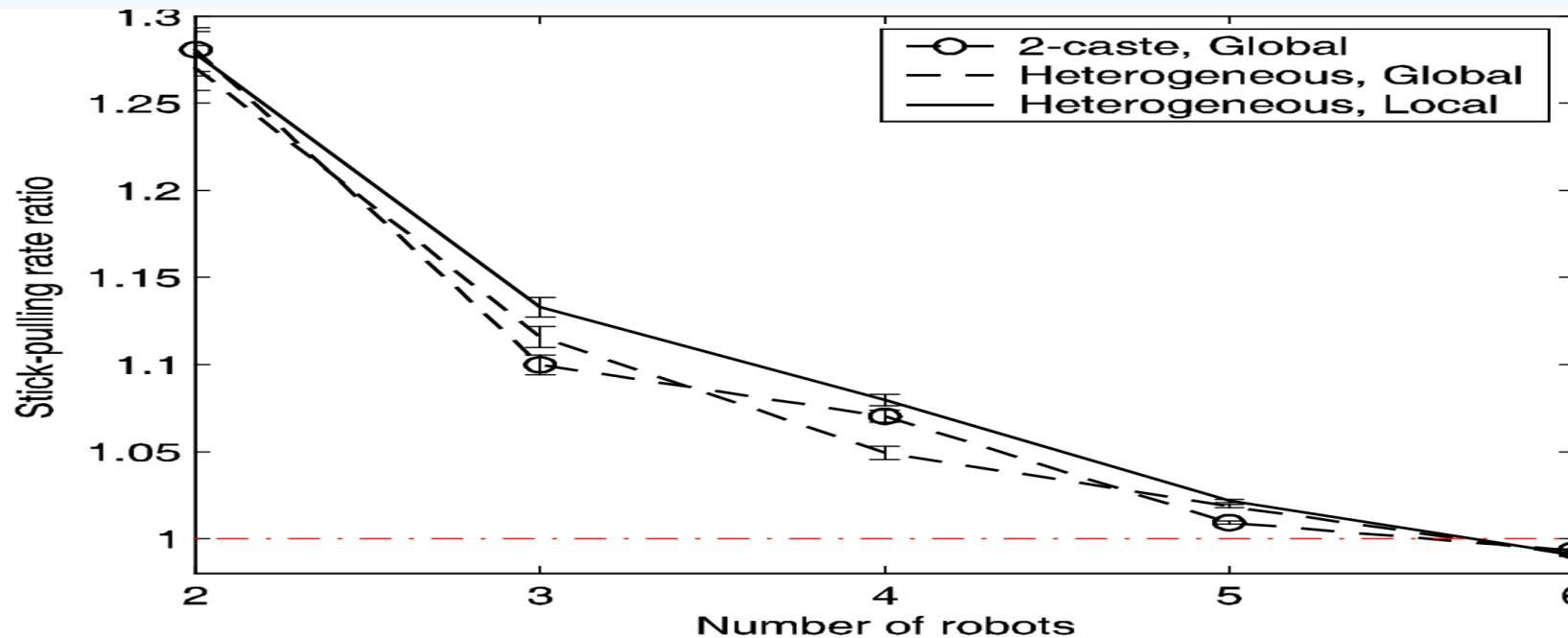
Политика разъезда роботов, идеально работавшая в симуляторе, на складе упирается в задержку радиоканала: сосед реагирует на полсекунды позже, чем ожидала политика, и согласованный манёвр рассыпается.

9 Многоуровневое моделирование роевой системы



Уровни абстракции роевой системы: от целевой системы через субмикроскопический и микроскопический к макроскопическому; на всех уровнях используются общие метрики.

9 Гетерогенное и гомогенное обучение: производительность



Notes:

- large window
- global evaluation
- local evaluation

Performance ratio between heterogeneous (full castes) and homogeneous groups AFTER learning

Отношение производительности гетерогенных (полная и двухкастовая) и гомогенных групп после обучения в зависимости от числа роботов; глобальная и локальная оценка.

Гомогенное или гетерогенное обучение?

Гомогенное (parameter sharing)

- Один общий набор параметров на всех
- Резко меньше размерность обучения
- Легко добавить робота в группу
- Специализация невозможна

Гетерогенное

- У каждого робота своя политика
- Возможны касты и разделение ролей
- Ценно для малочисленных групп
- Дороже, и выигрыш не гарантирован

? Как думаете?

В статье предложен новый сложный метод MARL. Он обходит QMIX на двух средах. Чего в такой работе не хватает, чтобы результату можно было верить?

A Сравнения с IPPO/MAPPO при одинаковом бюджете и настройке

B Более глубокой нейросети в критике

C Использования MADDPG вместо QMIX

D Большого числа агентов в эксперименте

Ответ: Сравнения с сильными простыми базовыми методами — IPPO и MAPPO — при равном бюджете взаимодействий, равной процедуре настройки и достаточном числе запусков.

- **Навигация флота:** склад, узкие проезды, конкуренция за перекрёстки
- Обучение в симуляторе по CTDE, исполнение — автономное, по локальным данным
- **Коллективный поиск и покрытие:** спасение, мониторинг, патрулирование БПЛА
- Группа сама вырабатывает пространственное разделение труда без планировщика
- **Тесная координация:** совместный перенос, кооперативная манипуляция, футбол роботов

Кейс

На складе роботы поначалу активно мешают друг другу, и лишь постепенно из обучения проступают устойчивые «правила движения»: приоритеты проезда, обгоны, разведение потоков по разным коридорам.

- Рост числа агентов раздувает совместные действия и вход критика
- Средства: parameter sharing, иерархии, подгруппы, механизмы внимания
- Обученная политика — статистический объект без гарантий поведения
- Это неприемлемо для роботов, работающих рядом с людьми
- Безопасный RL: constrained MDP, защитные оболочки (shielding), верификация сетей

Выводы

- 1 RL формализуется как MDP $\langle S, A, P, R, \gamma \rangle$; цель — максимум дисконтированной доходности
- 2 Уравнения Беллмана связывают ценности; $\pi(s) = \operatorname{argmax}_a Q(s, a)$
- 3 Q-обучение (off-policy) и SARSA (on-policy) учатся по TD-ошибке из опыта
- 4 DQN держится на буфере воспроизведения и целевой сети; градиент политики — для непрерывных действий
- 5 Многоагентность даёт нестационарность и проблему распределения заслуг
- 6 CTDE — доминирующая парадигма; VDN, QMIX, MADDPG, COMA, а IPPO и MAPPO — сильнейшие простые базовые методы
- 7 Практика упирается в воспроизводимость, бенчмарки, sim-to-real и безопасность

Вопросы для самопроверки

1. Чем RL отличается от обучения с учителем? В чём дилемма исследования и использования?
2. Определите MDP $\langle S, A, P, R, \gamma \rangle$ и поясните двоякую роль γ
3. Запишите уравнение Беллмана для $V_{\pi}(s)$ и уравнение оптимальности для $Q^*(s, a)$
4. Запишите правило Q-обучения. Что такое TD-ошибка и почему метод off-policy?
5. Чем SARSA отличается от Q-обучения и когда она предпочтительнее на работе?
6. Какие два механизма стабилизируют DQN и какие проблемы они решают?
7. Что такое нестационарность и проблема распределения заслуг в MARL?
8. Сформулируйте идею CTDE и назовите её практические ограничения
9. Какое свойство смешивающей сети QMIX обеспечивает согласованность $\arg\max$?
10. Почему IPPO и MAPPO обязательны как базовые линии в любом сравнении методов?